

Stoquify AI Agents and Skills Runtime Note

Date: 2026-07-16

Topic: How the project's AI agents and skills serve users when the LLM is not bundled with the project

Short Answer

The LLM is not inside the Stoquify project. The project should treat the LLM as an external or separately hosted reasoning service reached through a model gateway. Stoquify keeps the valuable and defensible parts: tenant context, service-owned tools, permissions, evidence, workflows, approval rules, and audit trails.

Today, the repository contains agent and skill definitions, schemas, registries, and governance contracts. Those files are not enough by themselves to serve product users. A production Copilot runtime still has to be built and deployed to load those definitions, call an approved model provider, validate outputs, execute allowlisted tools, and persist evidence.

Practical Runtime Flow

1. A user asks a question, a schedule fires, or a business event creates a case.
2. Stoquify resolves tenant, organization, actor, role, location, module, route, locale, feature flag, and correlation context.
3. Permission, entitlement, redaction, data-classification, and freshness checks run before retrieval.
4. Approved Stoquify services and read models retrieve the minimum necessary evidence.
5. The model is called through a model gateway with bounded instructions, schemas, tool limits, and evidence references.
6. The model returns structured output, not direct database mutations.
7. Stoquify validates the output against versioned schemas and policy.
8. Read-only work returns evidence-linked answers with uncertainty and next safe action.
9. Write-capable work becomes a draft, approval request, or controlled service command depending on risk.
10. Any actual write runs only through deterministic Stoquify services with idempotency, authorization, approval, fresh authentication where required, and audit.

Required Production Pieces

- **Agent control plane:** stores agent definitions, versions, runs, steps, tool calls, evidence links, approvals, feedback, costs, and incidents.
- **Model gateway:** routes to approved providers or self-hosted models, applies budgets, handles fallback, records model/version/latency/cost, and keeps provider choice portable.
- **Tool gateway:** exposes allowlisted Stoquify service commands and read models. Agents must not write directly to Prisma or bypass service-owned business rules.
- **Skill registry:** loads versioned skills, schemas, permitted tools, evidence contracts, risk class, timeout, retry policy, and release state.

- **Policy layer:** enforces tenant isolation, RBAC, module entitlement, redaction, fresh-auth requirements, maker-checker rules, and approval policy.
- **Background worker:** handles scheduled briefs, event-driven cases, retries, long-running work, notifications, and resumable runs.
- **Observability plane:** records trace IDs, policy denials, model/tool versions, redaction events, token/cost usage, latency, failures, approvals, user corrections, and business outcomes.

What Happens If the LLM Is Unavailable

Stoquify should degrade safely. Deterministic dashboards, action queues, alerts, rule-based checks, cached briefs, and existing workflows can still work. Agent reasoning, summarization, planning, and natural-language explanation should return a clear unavailable, degraded, blocked, or retryable state.

The system should never pretend that an agent completed reasoning or performed a side effect when the model or tool path failed.

Ownership Boundaries

- **Stoquify services own truth:** balances, stock quantities, ledger postings, payments, payroll calculations, permissions, entitlements, and statutory states remain service-owned.
- **Agents coordinate:** they interpret user intent, select approved skills, retrieve evidence, explain, prioritize, draft, and route work.
- **Skills constrain behavior:** each skill is a narrow, versioned, testable capability with inputs, outputs, evidence rules, permissions, and safe failure states.
- **The model reasons:** it helps summarize, classify, plan, and explain, but it does not receive unlimited data or authority.
- **Humans approve high-risk work:** financial, payroll, compliance, entitlement, destructive, irreversible, and statutory actions require the existing product controls and human authorization.

Deployment Interpretation

The right architecture is not a swarm of bots and not a branded wrapper around a public model. It is a controlled operating Copilot:

```

User, schedule, or event
|
Stoquify Copilot runtime
|
Context resolver + policy gate
|
Evidence retrieval from service-owned read models
|
Model gateway for bounded reasoning
|
Schema and policy validation
|
Read response, draft, approval request, or controlled service command
|

```

Evidence, audit, telemetry, and outcome record

Near-Term Recommendation

Build the first production slice as a read-only Stoquify Command Agent plus a shared Exception and Action Orchestrator. Keep cash/reconciliation and inventory workflows as the first domain pilots. Do not start with autonomous financial posting, payroll approval, statutory filing, period close, RBAC changes, payment release, stock write-off, or direct database writes.

The clean mental model is:

Skills are the playbooks, agents are bounded actors, the LLM is rented or separately hosted reasoning, and Stoquify services remain the source of truth.

Repository Evidence Used

- docs/copilot/stoquify-agent-skill-definition-suite/reports/completion-report.md
- docs/agents and skills/STOQUIFY_ENTERPRISE_AGENT_AND_SKILL_SYSTEM_ASSESSMENT_2026-07-15.md
- docs/copilot/STOQUIFY_FUTURE_READY_COPILOT_BACKBONE_ARCHITECTURE_PROMPT.md
- docs/copilot/stoquify-agent-skill-definition-suite/registry/capability-registry.md
- docs/copilot/stoquify-agent-skill-definition-suite/contracts/agent-output.schema.json
- docs/copilot/stoquify-agent-skill-definition-suite/contracts/risk-autonomy-policy.md
- docs/copilot/stoquify-agent-skill-definition-suite/contracts/controlled-write-protocol.json